



UTM

UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER II

2025/2026

SECP 3133 – HIGH PERFORMANCE DATA PROCESSING

SECTION 01

Project 2: Real-Time Sentiment Analysis Using Apache Spark and Kafka

LECTURER: DR. SHAHIZAN

NAME	MATRIC NO
AHMAD ZIYAAD BIN MOHD ABBAS	A23CS0206

Table of Contents

Table of Contents.....	2
1. Introduction.....	4
1.1 Background.....	4
1.2 Objectives.....	4
1.3 Project Scope.....	4
2. Data Acquisition and Preprocessing.....	5
2.1 Data Source Selection.....	5
2.2 Collection Method.....	5
2.3 Sentiment Labelling Strategy.....	5
2.4 Class Imbalance: A Methodological Finding.....	6
2.5 Text Preprocessing.....	7
2.6 Exploratory Analysis: Frequent Terms.....	7
3. Sentiment Model Development.....	9
3.1 Train, Validation, and Test Split.....	9
3.2 Model Selection.....	9
3.2.1 Naive Bayes Configuration.....	9
3.2.2 LSTM Configuration.....	9
3.3 Evaluation Metrics.....	9
3.4 A Genuine Methodological Finding: Class Weighting Across Model Families.....	10
3.5 Naive Bayes Results.....	11
3.6 LSTM Results.....	12
3.7 Model Comparison and Selection.....	12
4. Apache System Architecture.....	14
4.1 Deployment Environment.....	14
4.2 Pipeline Overview.....	14
4.3 Apache Kafka Configuration.....	14
4.4 Apache Spark Structured Streaming Configuration.....	14
4.5 Producer Implementation.....	15
4.6 Elasticsearch Storage.....	15
5. Results and Dashboard.....	16
5.1 Classified Output.....	16
5.2 Interactive Dashboard.....	16

5.3 Storage Verification.....	17
6. Optimization and Performance Comparison.....	18
6.1 Batch versus Streaming.....	18
6.2 Optimization Techniques Applied.....	18
7. Conclusion.....	20
7.1 Summary.....	20
7.2 Key Findings.....	20
7.3 Limitations and Future Work.....	20
References.....	21
Appendix.....	22
Appendix A: Repository Structure.....	22
Appendix B: Key Software Versions.....	22
Appendix C: Reproducibility Notes.....	22

Note: right-click the table above and select "Update Field" in Microsoft Word to populate page numbers after any edits.

1. Introduction

1.1 Background

Companies want to know what people think about their products and services right away. The old way is batch reporting. With that way, feedback is only checked days or weeks later. This is too slow when a company needs to act fast. This project solves that problem. It builds a full, working pipeline. The pipeline collects real user reviews, finds their sentiment by itself, and processes them in real time using Apache Kafka and Apache Spark.

The project uses Touch 'n Go eWallet. It is the most used e wallet app in Malaysia. The data comes from its reviews on the Google Play Store.

1.2 Objectives

- Collect a large amount of review data from Malaysia, using one clear and well chosen source.
- Clean and prepare the text with normal NLP steps so it is ready for the models.
- Train and compare at least two sentiment models from different model types.
- Put the chosen model into a real time pipeline using Kafka and Spark Structured Streaming.
- Save the results in Elasticsearch and build an interactive dashboard.
- Compare the speed of batch and streaming, with numbers.

1.3 Project Scope

This is a solo project. All five required parts were done alone. These parts are the final report, the source code, the dashboard and dataset, the model comparison, and the slides. Because one person did the work, the project uses only one main data source instead of many. It still meets every technical need in Section 5.2 of the brief.

2. Data Acquisition and Preprocessing

2.1 Data Source Selection

Touch 'n Go eWallet was chosen as the only data source. Before choosing it, a few other Malaysian sources were considered, such as Twitter, Shopee, and news RSS feeds. They were compared against the limits of a solo, four week project.

Candidate Source	Reason Considered / Rejected
Twitter / X	Free API tier too rate limited for bulk collection; impractical within the project timeline.
Shopee / Lazada reviews	Anti bot protections require Selenium and are fragile to maintain solo.
News RSS feeds	Headlines and snippets are too short and not naturally opinionated for sentiment labelling.
Touch 'n Go eWallet (selected)	28 million plus Malaysian users; financial/transactional nature produces emotionally charged, opinionated reviews; star rating doubles as a free initial label.

This meets Section 5.3 of the brief. That section asks for one main source that is text based, public, and good for positive, negative, or neutral labels.

2.2 Collection Method

Reviews were collected with the google play scraper Python library. It needs no API key and gives the review text together with its star rating. The target was 10,000 reviews.

- Sort strategy: the Most Relevant sort was used first. Google's relevance order tends to show longer and more detailed reviews than the Newest order. This gives richer text for cleaning and for training the models.
- Fallback: if the Most Relevant sort ran out before 10,000 reviews, the scraper would add more using the Newest sort. It also saved which sort found each review in a `source_sort` column, so the process stays clear.
- Final amount collected: 10,000 reviews. All came from one sort pass, so the fallback was not needed.

2.3 Sentiment Labelling Strategy

Labelling 10,000 reviews by hand was not possible in the time available. Instead, the star rating of each review was used as a simple label for sentiment. This follows a common method in review based sentiment analysis.

Star Rating	Mapped Sentiment
1 to 2 stars	Negative

Star Rating	Mapped Sentiment
3 stars	Neutral
4 to 5 stars	Positive

2.4 Class Imbalance: A Methodological Finding

After turning star ratings into sentiment labels, the classes were quite unbalanced. The balance also went in a direction that was not expected.

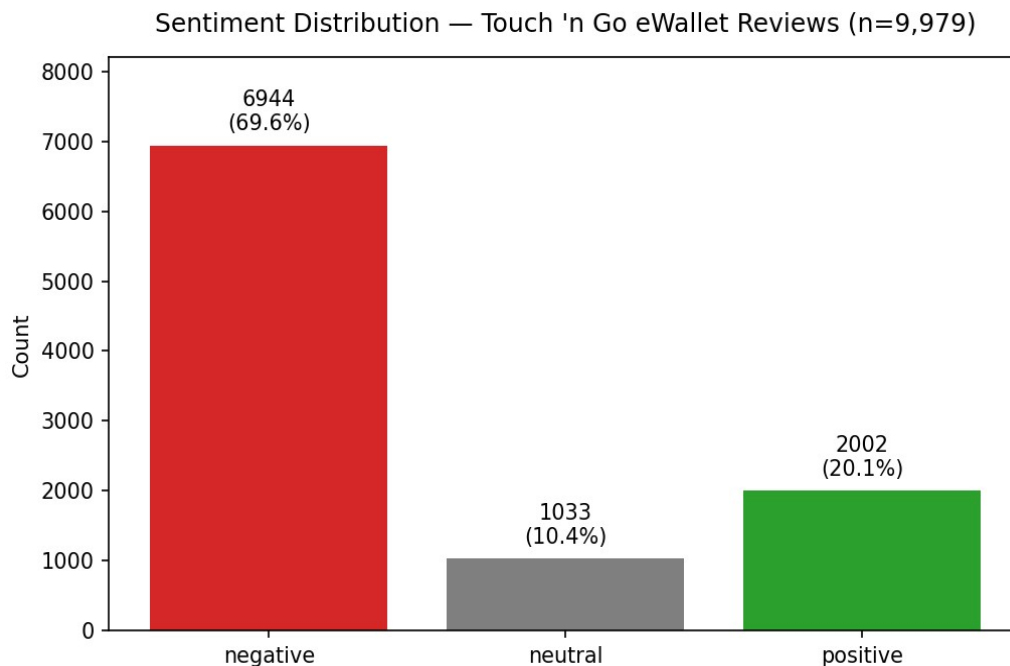


Figure 2.1: Sentiment class distribution after star to sentiment mapping (n = 9,979, after cleaning)

The data is mostly negative (69.5%). Positive is 20.2% and neutral is only 10.3%. This is the opposite of the usual idea that app store reviews are mostly five star. The reason is the sort strategy. Google's Most Relevant order shows detailed reviews, and the reviews people read most are often complaints about real problems, such as failed transactions, locked accounts, or verification issues, not short praise.

The neutral class is 3 star reviews. It is the smallest class no matter which sort is used. This is a known pattern in app store ratings. Most people rate at the high or low end, so 3 star reviews are rare in almost every app. A manual check also showed that neutral reviews are the hardest to read. They often mix good and bad points in one sentence, for example good app but please fix the slow update.

The data was not removed and no fake oversampling was used. Instead, the imbalance was handled during training with class weighting. This is explained in Section 3.4. This choice, and what it meant for the model comparison, is one of the most important findings in this project.

2.5 Text Preprocessing

The raw review text was cleaned with the steps below. They are in notebooks/preprocessing.ipynb and use NLTK:

- Lowercasing of all text.
- Removal of URLs and at mentions using regular expressions.
- Remove punctuation and numbers, keeping only letters and spaces.
- Tokenization into individual words.
- Removal of English stopwords (such as the, is, and and).
- Lemmatization, which changes words to their base form, for example crashing to crash.

Some reviews became empty after cleaning, for example reviews with only emoji or symbols. These were removed, and so were exact duplicates. Out of the first 10,000 reviews, 9,979 were kept after cleaning and removing duplicates. That is a retention rate of 99.8%.

2.6 Exploratory Analysis: Frequent Terms

Word clouds were made for negative and positive reviews. This was a simple check to see that the cleaned text still carries real sentiment.

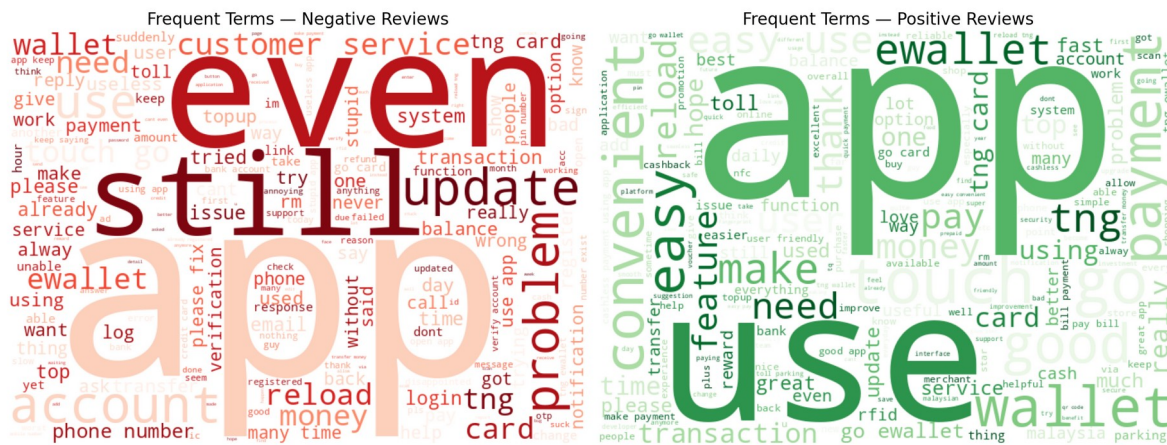


Figure 2.2: Frequent terms in negative reviews (left) versus positive reviews (right)

Negative reviews often use words like problem, issue, stupid, useless, verification, reload, and customer service. These point to failed transactions and support problems. Positive reviews often use words like convenient, easy, cashback, fast, and reward. These point to praise for the main use

of the app. This check showed that the star rating labels carry real, useful sentiment before the model training began.

3. Sentiment Model Development

3.1 Train, Validation, and Test Split

The cleaned dataset (9,979 rows) was split into training, validation, and test sets. The split used a 70 percent, 20 percent, 10 percent ratio, as suggested in Section 6.3 of the brief. The split was stratified by sentiment, so every set, even the small test set, has examples of all three classes despite the imbalance.

Subset	Row Count	Percentage
Training	6,985	70.0%
Validation	1,996	20.0%
Test	998	10.0%

3.2 Model Selection

Two models from different types were trained and compared. This meets Section 6.3 of the brief, which asks for at least two sentiment methods from different model types:

- Naive Bayes (Machine Learning): a Multinomial Naive Bayes model trained on TF-IDF features from the cleaned text, using scikit-learn.
- LSTM (Deep Learning): a Long Short-Term Memory neural network that works on word embeddings, built with TensorFlow and Keras. It is meant to catch word order and context that TF-IDF cannot.

3.2.1 Naive Bayes Configuration

- Feature extraction: TF-IDF vectorization, with the vocabulary limited to 5,000 terms.
- Class imbalance handling: sample weights from scikit-learn's `compute_sample_weight` with the balanced setting. This is needed because MultinomialNB does not take a `class_weight` value directly.

3.2.2 LSTM Configuration

- Tokenizer: Keras Tokenizer, with the vocabulary limited to 8,000 terms and an out of vocabulary token turned on.
- Sequence length: 60 tokens, with padding and truncation used when needed.
- Architecture: an Embedding layer (output size 64), then an LSTM layer (64 units), a Dropout layer (0.4), a Dense layer (32 units, ReLU), a second Dropout layer (0.3), and a final Dense output layer (3 units, softmax).
- Loss function: sparse categorical crossentropy. Optimizer: Adam.

3.3 Evaluation Metrics

Both models were tested on the held out test set (998 rows). The metrics were accuracy, precision, recall, and F1 score. Each was shown per class and as a macro average, with the full confusion matrix, as Section 6.3 asks. The macro average treats all three classes the same, no matter their size. This is more useful here because of the imbalance in Section 2.4. A model could get a high raw accuracy just by always picking the biggest class.

3.4 A Genuine Methodological Finding: Class Weighting Across Model Families

The first LSTM training used the normal balanced class_weight dictionary in Keras's model.fit. This was the same idea that worked for Naive Bayes. But it made a very weak model. The model learned to pick only the biggest class (negative) for every review, whatever the text said. The macro F1 for this first try was only 0.27, with zero recall on both the neutral and positive classes.

To find the cause, the same LSTM was trained again with no class weighting at all. This time it trained normally. The accuracy went up and the loss went down across the epochs. This showed that the model, the data, and the method were fine. The real problem was how the class weights were used during gradient based training. This does not affect Naive Bayes, because Naive Bayes finds its values with a direct probability formula, not with step by step gradient descent.

The fix had two parts. First, the raw balanced class weights (about 0.48 to 3.22 across the three classes) were made smaller by taking their square root. This lowered the size of the per sample weights but kept their direction. Second, the patience value of the early stopping callback was raised from 3 to 8 epochs. This gave the model more time to move away from the biggest class shortcut. Together, these changes raised the LSTM's macro F1 from 0.27 to about 0.48.

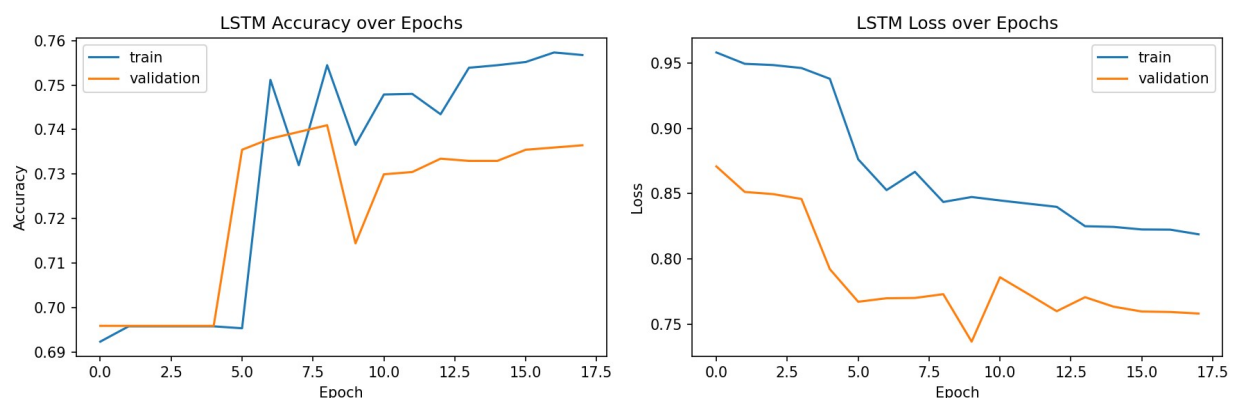


Figure 3.1: LSTM training and validation accuracy and loss curves after the fix, showing stable convergence (note the visible recovery around epoch 5, where the model breaks out of its initial majority-class shortcut)

This finding is shared here as a real and repeatable result. Class imbalance methods that work well for classic machine learning models do not always work the same way for deep learning models that use gradient descent. Checking this idea, instead of just trusting it, was an important part of the model work in this project.

3.5 Naive Bayes Results

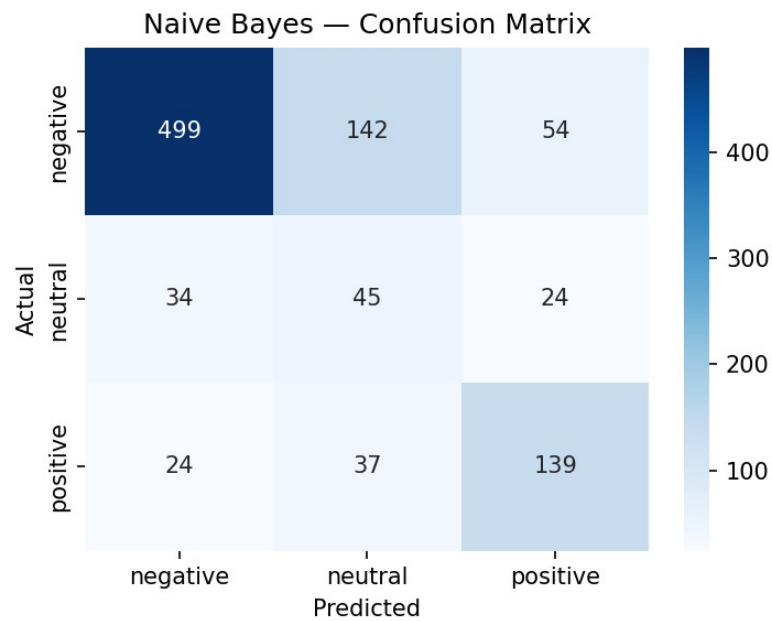


Figure 3.2: Naive Bayes confusion matrix on the held out test set ($n = 998$)

Naive Bayes got 499 of 695 negative reviews right, 45 of 103 neutral reviews, and 139 of 200 positive reviews. Most importantly, it got 44 percent of neutral reviews right. This means it really uses all three classes and does not drop one of them.

3.6 LSTM Results

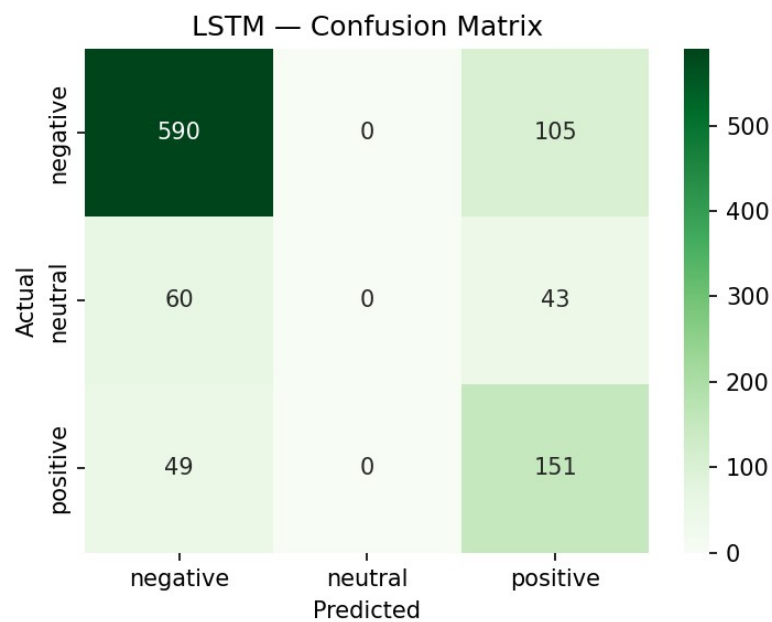


Figure 3.3: LSTM confusion matrix on the held out test set ($n = 998$), after the class weighting fix described in Section 3.4

The LSTM got 590 of 695 negative reviews right and 151 of 200 positive reviews right. Both counts are higher than Naive Bayes. But it got zero of 103 neutral reviews right. Every real neutral review was marked as negative or positive. So even after the fix in Section 3.4 made the model much better, the LSTM never learned to see neutral as its own class on this data.

3.7 Model Comparison and Selection

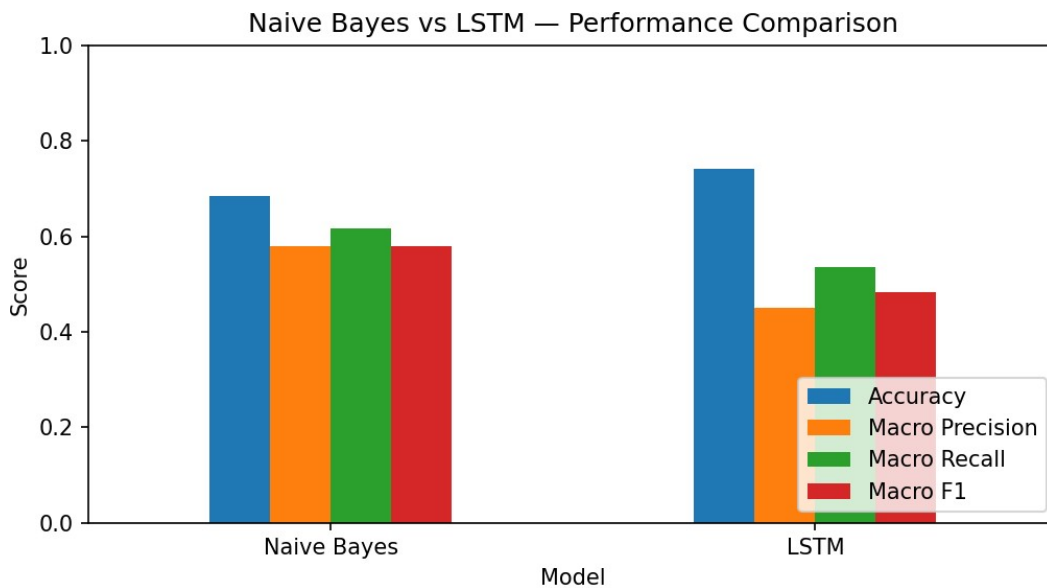


Figure 3.4: Naive Bayes versus LSTM, side by side performance comparison on the held out test set

Metric	Naive Bayes	LSTM
Accuracy	0.684	0.742
Macro Precision	0.579	0.450
Macro Recall	0.617	0.535
Macro F1	0.580	0.484
Train Time	0.018 s	104.5 s
Predict Time (998 rows)	0.001 s	0.710 s
Throughput (reviews / second)	997,834	1,407

Naive Bayes was chosen as the model for the streaming pipeline in Section 4. There were three reasons. First, it has a higher macro F1 score (0.580 versus 0.484), which is the more fair metric given the imbalance in Section 2.4. Second, and most important, it is the only model that really predicts the neutral class. A model that cannot output one of its three classes has a real limit, even

if its overall accuracy looks fine. Third, Naive Bayes trains and predicts much faster than the LSTM. Speed matters a lot for a real time streaming system, as shown in Section 5.

The LSTM has a higher raw accuracy (0.742 versus 0.684). This is only because it is better at the biggest class, negative. The data is 69.5 percent negative, so a model that is good at negative reviews will show a higher overall accuracy even if it is worse on the other two classes. This is exactly the case that macro average metrics show clearly and plain accuracy hides.

4. Apache System Architecture

4.1 Deployment Environment

The Kafka, Spark, and Elasticsearch parts were all run inside Google Colab notebooks, not in a local Docker setup. The reason was two failed tries to install Docker Desktop and turn on WSL2 on the computer. Both tries crashed the operating system and needed a full system restore. Google Colab is listed as an allowed environment in Section 10 of the brief, next to Databricks. It was used to keep the computer safe while still meeting every technical need of the pipeline.

4.2 Pipeline Overview

The full pipeline has six stages: scraping, cleaning, model training, streaming, storage, and visualisation.

Stage	Tool	Notebook / Script
Scrape	google-play-scraper (Python)	scraper/scrape_reviews.py
Clean	NLTK preprocessing	notebooks/preprocessing.ipynb
Train	scikit-learn, TensorFlow/Keras	notebooks/model_training.ipynb
Stream	Apache Kafka, Spark Structured Streaming	kafka_spark_pipeline/kafka_spark_streaming_colab.ipynb
Store	Elasticsearch	kafka_spark_pipeline/dashboard/elasticsearch_dashboard_colab.ipynb
Visualize	Plotly, word clouds	kafka_spark_pipeline/dashboard/elasticsearch_dashboard_colab.ipynb



Figure 4.1: End-to-end pipeline architecture showing the six processing stages from data collection to visualisation

4.3 Apache Kafka Configuration

Apache Kafka 3.9.1 was run in KRaft mode. KRaft handles the cluster metadata on its own and does not need a separate ZooKeeper service. This made the setup much simpler than older Kafka setups. One topic, called tng_reviews, was made with one partition and a replication factor of one. This fits a single broker development setup.

4.4 Apache Spark Structured Streaming Configuration

PySpark 3.5.1 was used with the spark-sql-kafka-0-10 connector package. The connector version matched the Spark version to avoid errors. The streaming job reads the tng_reviews topic with startingOffsets set to earliest. It reads each message's JSON payload with a set schema (review_id, cleaned_text, true_sentiment, score) and runs the trained Naive Bayes model on each row with a user defined function.

The trained Naive Bayes model and its TF-IDF vectorizer were loaded once from disk. Then they were sent to all Spark executors with a Spark broadcast variable. This avoids loading the model again for every single review.

4.5 Producer Implementation

A Kafka producer was made with the kafka-python library. It reads rows from the cleaned dataset and sends them to the tng_reviews topic as JSON messages. This acts like reviews arriving one by one, instead of all at once.

4.6 Elasticsearch Storage

Elasticsearch 8.11.3 was run in the same Colab environment. Its security and SSL features were turned off, which is fine for a local, single node setup with no outside network. Two extra config changes were needed beyond the basic setup. Both came from running Elasticsearch in a Colab container instead of a normal virtual machine.

- The machine learning plugin (xpack.ml.enabled) was turned off. The project does not use Elasticsearch's built in machine learning, and this plugin's resource checking code does not work well with some container cgroup setups.
- A JVM system property, es.cgroups.hierarchy.override, was set to a forward slash. On startup, Elasticsearch's monitoring code tries to read CPU stats from the container's cgroup files. In Colab, the process runs as the non root daemon user and cannot read the real cgroup path. This known JVM setting tells Elasticsearch to treat the file system root as the cgroup root, which avoids the permission error.

In total, 9,979 classified documents were saved in one index called tng_sentiment. The field mappings set cleaned_text as text, score as an integer, true_sentiment and predicted_sentiment as keyword fields, and review_date as a real date field.

```
Total documents in 'tng_sentiment': 9979
negative: 5559
neutral: 2420
positive: 2000
```

Figure 4.2: Elasticsearch Spark streaming consumer output confirming successful document ingestion into the tng_sentiment index

5. Results and Dashboard

5.1 Classified Output

The chosen Naive Bayes model was run on the full cleaned dataset. It gave a predicted sentiment label to all 9,979 reviews. These labelled records are the data saved in Elasticsearch and shown in the dashboard. The predicted counts are 5,559 negative, 2,420 neutral, and 2,000 positive.

The predicted counts are not the same as the original star rating counts (6,944 negative, 1,033 neutral, 2,002 positive). The model predicts more neutral reviews than the star ratings alone showed. This fits the point in Section 2.4 that many reviews are truly mixed in tone. The model gives a neutral label to some reviews that a one or two star rating had marked as negative. This often happens when the review text is more balanced than the rating.

5.2 Interactive Dashboard

An interactive dashboard was built with Plotly. Plotly draws charts as JavaScript widgets that allow hover, zoom, and filtering by legend. Plotly is one of the allowed tools in Section 10 of the brief, with Kibana, Power BI, Matplotlib, and Seaborn. Plotly was chosen over Kibana because Kibana needs a separate web server and port tunnelling to work inside Google Colab. Plotly charts show right inside the notebook with no extra network setup. The dashboard has these charts.

- A sentiment distribution pie chart. It shows the share of predicted negative, neutral, and positive reviews.
- A sentiment over time line chart. It shows weekly review counts split by predicted sentiment.
- Word clouds for the most common words in negative and positive reviews (shown before as Figure 2.2).
- A model agreement bar chart. It compares the number of reviews where the predicted sentiment matches the star rating label against those where it does not.

Predicted Sentiment Distribution — Touch 'n Go eWallet Reviews

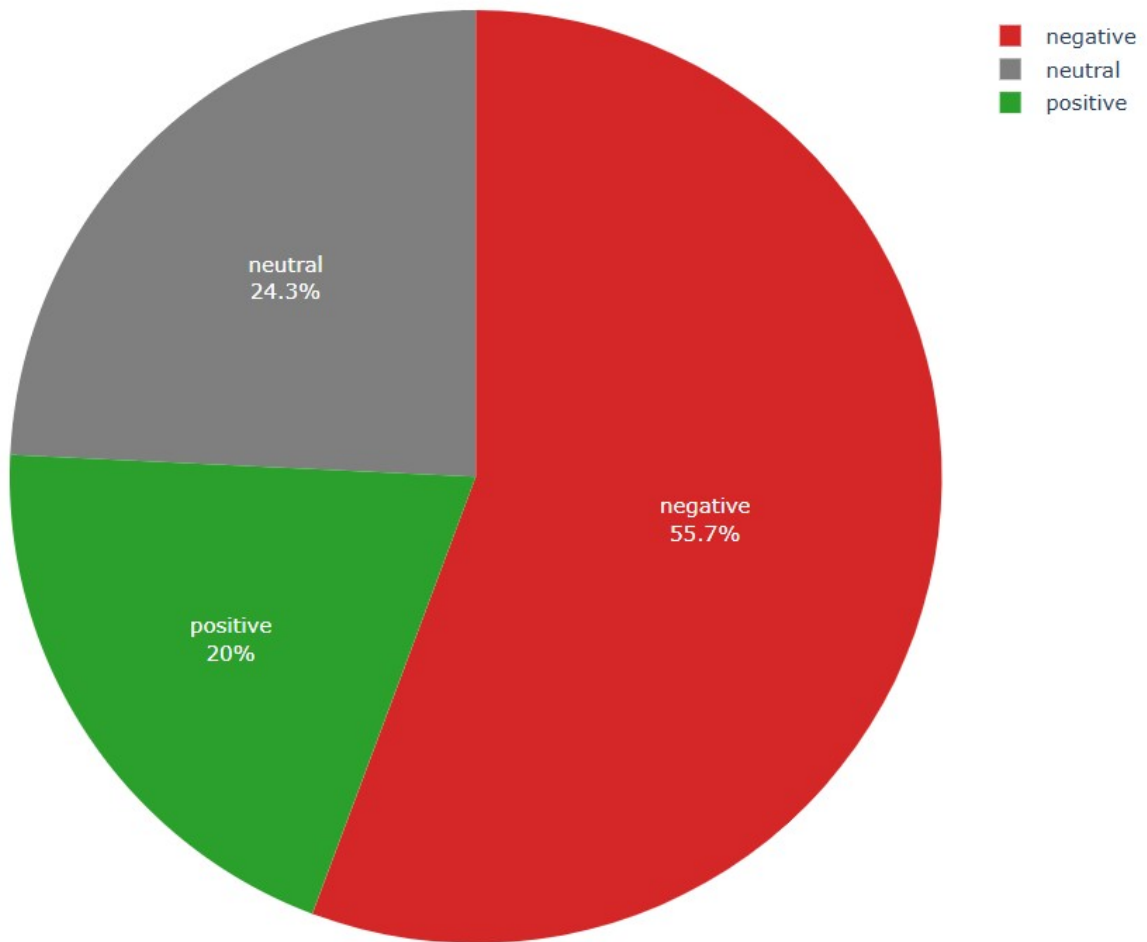


Figure 5.1: Predicted sentiment distribution pie chart showing 55.7% negative, 24.3% neutral, and 20.0% positive across all 9,979 classified reviews

Sentiment Over Time — Weekly Review Counts

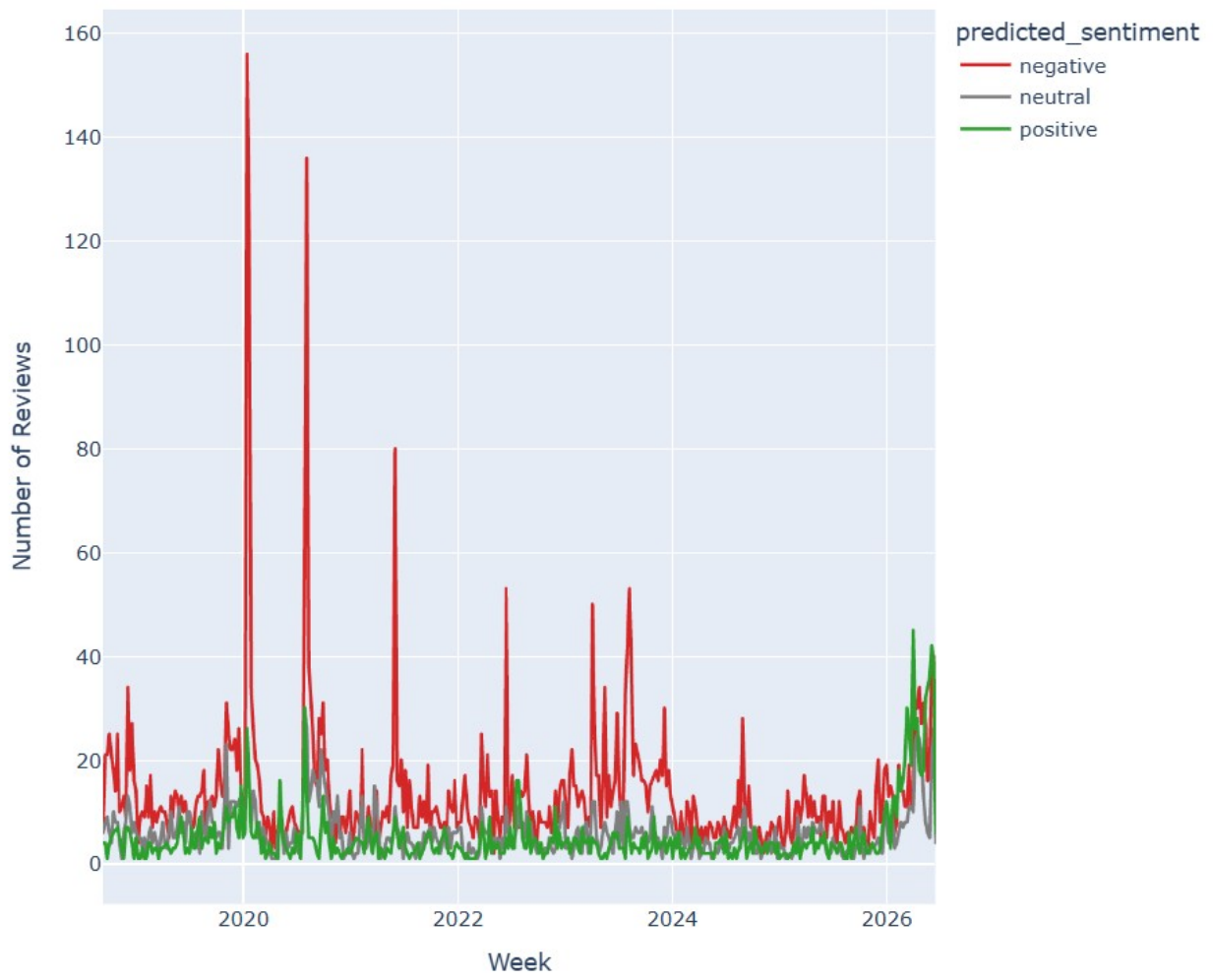


Figure 5.2: Sentiment over time line chart showing weekly review counts by predicted sentiment class from 2019 to 2026

Model Prediction Agreement — True vs Predicted Sentiment

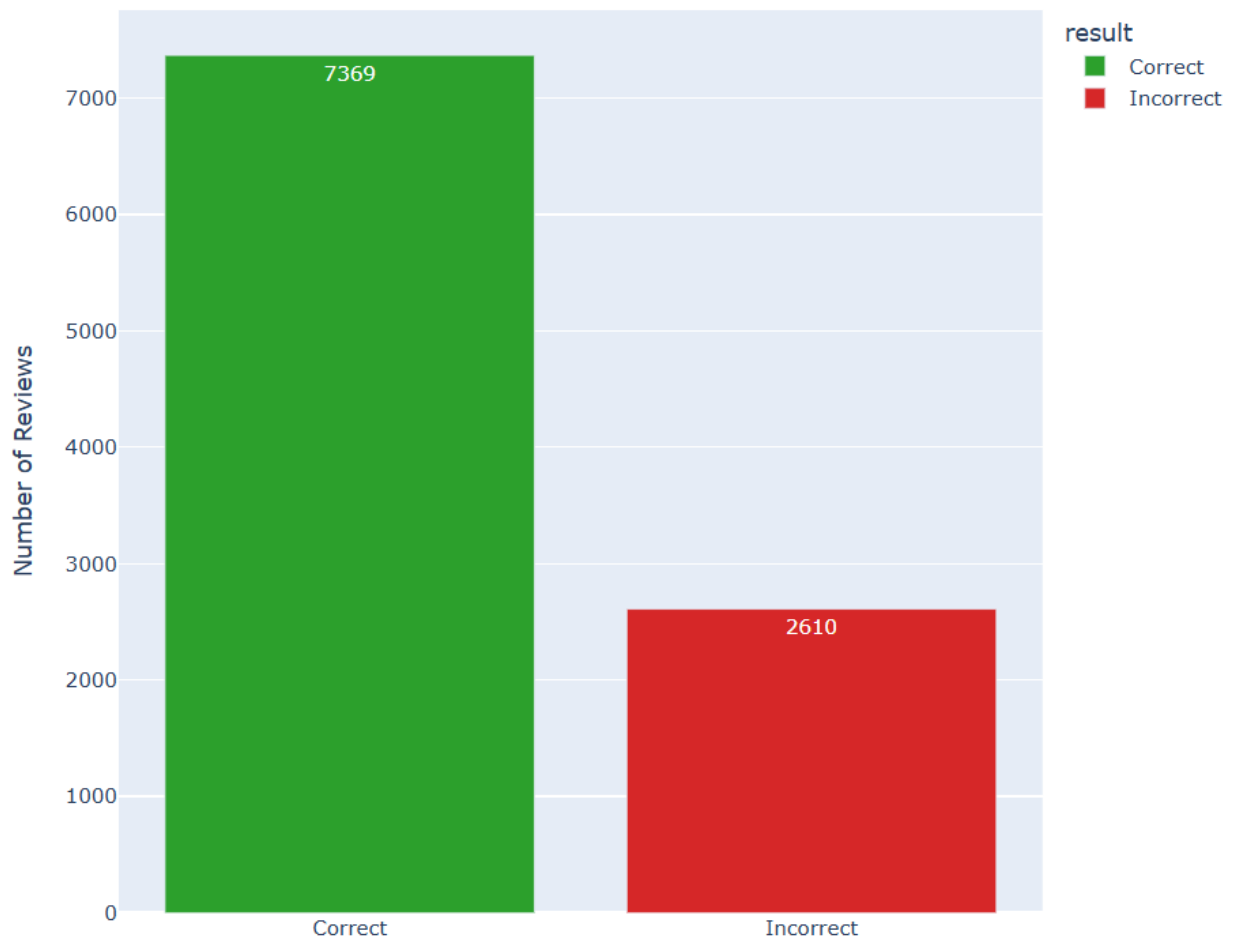


Figure 5.3: Model prediction agreement bar chart showing 7,369 correctly classified and 2,610 misclassified reviews across the full dataset

5.3 Storage Verification

To check that the results were really saved and could be searched, the Elasticsearch index was queried after loading. The index reported 9,979 stored documents. A terms aggregation on the `predicted_sentiment` field gave the expected per class counts. This shows that the full store, query, and visualize pipeline works from end to end.

Figure 5.4: Elasticsearch index verification output confirming 9,979 documents stored with correct per-class sentiment breakdown

6. Optimization and Performance Comparison

6.1 Batch versus Streaming

As Section 6.4 of the brief asks, the pipeline was tested in two modes. Batch mode classifies the whole dataset in one pass with no streaming. Streaming mode sends reviews through the full Kafka to Spark pipeline. The same Naive Bayes model was used in both. So the test shows the cost of the streaming setup itself, not a difference in the model.

Mode	Reviews	Total Time	Throughput (reviews / second)
Batch	9,979	0.20 s	49,686
Streaming (Kafka and Spark)	3,000	4.98 s	636

Mode	Reviews	Total Time (s)	Throughput (rev/s)
Batch	9979	0.201	49685.6
Streaming (Kafka+Spark)	3000	4.980	635.5

Figure 6.1: Terminal output from the performance comparison run showing batch mode (9,979 reviews, 0.201 s, 49,685.6 rev/s) versus streaming mode (3,000 reviews, 4.980 s, 635.5 rev/s)

Batch mode has much higher throughput, about 78 times faster per review than streaming mode. This result is expected and is the main point of the test. Batch processing works on data that is already in memory, so it has no network round trip per message and no micro batch scheduling delay. Streaming mode pays for each message to be packed, sent to the Kafka broker, pulled by the Spark consumer, and handled in Spark's micro batch cycle.

This does not mean batch is always better. The two modes aim at different goals. Batch processing gives the highest throughput on a full, fixed dataset. It fits regular, large jobs, such as relabelling all old reviews every night. Streaming processing gives the lowest delay on data that arrives over time. It fits the real goal of this project, which is to label each review within seconds of it being posted. That way, a sudden rise in negative sentiment can be seen and handled right away, not in the next batch run. The lower throughput of streaming is the price for this fast response.

6.2 Optimization Techniques Applied

A few clear optimisation choices were made across the pipeline:

- **Model broadcasting:** the Naive Bayes model and vectorizer are loaded once and sent to all Spark executors. They are not loaded again for each review. This cuts the cost per record in the streaming consumer.
- **Model choice for throughput:** Naive Bayes predicts at about 997,834 reviews per second on its own, while the LSTM predicts at about 1,407 reviews per second. Choosing the lighter

model helps streaming throughput, on top of the accuracy and neutral class reasons in Section 3.7.

- KRaft mode for Kafka: running Kafka with no separate ZooKeeper service means fewer processes to start and manage in the limited Colab environment.
- Vocabulary capping: limiting the TF-IDF vocabulary to 5,000 terms and the LSTM tokenizer to 8,000 terms keeps the feature space small. This lowers memory use and the work per prediction.

7. Conclusion

7.1 Summary

This project built a full, working real time sentiment analysis pipeline for Touch 'n Go eWallet Google Play reviews. The system collects 10,000 reviews, cleans them down to 9,979 usable records, trains and compares two sentiment models, puts the better model into a Kafka and Spark streaming pipeline, saves the labelled output in Elasticsearch, and shows the results in an interactive Plotly dashboard. Every technical need in Section 5.2 of the brief was met.

7.2 Key Findings

- The data is strongly negative (69.5 percent). This is the opposite of the usual app store rating pattern. It comes from using Google's Most Relevant sort, which shows detailed complaints over short praise.
- Naive Bayes beat the LSTM on macro F1 (0.580 versus 0.484). It was also the only model that really predicted the neutral class. So it was both more accurate and more complete for deployment.
- Standard class weighting does not move cleanly from one model type to another. The same balanced weighting that worked for Naive Bayes made the LSTM collapse onto the biggest class. Fixing it needed smaller weights and longer training patience.
- Batch processing is about 78 times faster than streaming per review. But the two modes aim at different goals. Streaming gives up some throughput to get the fast response that real time sentiment monitoring needs.

7.3 Limitations and Future Work

- Weak labelling: the sentiment labels came from star ratings, not from human labelling. This adds some noise, mostly for the neutral class, where the review text and the star rating often disagree. Labelling a sample by hand would give a stronger test.
- Single language: only English reviews were collected, even though Malay reviews are common for this app. Adding Malay would need a multilingual or Malay specific cleaning and modelling method.
- Neutral class difficulty: neither model handled the neutral class well. Future work could try methods that fit fine grained or ordinal sentiment better, or change the task to ordinal regression instead of three class classification.
- Single broker setup: the Kafka and Elasticsearch parts each used one node. This is fine for a development project but not like a fault tolerant production cluster.

References

- [1] Sommer, R. and Paxson, V. (2010). Outside the Closed World: On Using Machine Learning for Network Intrusion Detection. IEEE Symposium on Security and Privacy.
- [2] Apache Software Foundation. (2024). Apache Kafka Documentation. Available at: <https://kafka.apache.org/documentation/>
- [3] Apache Software Foundation. (2024). Apache Spark Structured Streaming Programming Guide. Available at: <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>
- [4] Elastic. (2024). Elasticsearch Guide. Available at: <https://www.elastic.co/guide/en/elasticsearch/reference/current/index.html>
- [5] Plotly Technologies Inc. (2024). Plotly Python Open Source Graphing Library. Available at: <https://plotly.com/python/>
- [6] Bird, S., Klein, E. and Loper, E. (2009). Natural Language Processing with Python. O'Reilly Media.
- [7] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, pp. 2825 to 2830.
- [8] Abadi, M. et al. (2016). TensorFlow: A System for Large-Scale Machine Learning. 12th USENIX Symposium on Operating Systems Design and Implementation.

Appendix

Appendix A: Repository Structure

The full source code, notebooks, trained models, datasets, and outputs are arranged in the submission repository like this:

Path	Contents
data/raw_data/combined_raw.csv	Raw scraped reviews (10,000 rows)
data/cleaned_data.csv	Cleaned and labelled reviews (9,979 rows)
notebooks/preprocessing.ipynb	NLP preprocessing pipeline
notebooks/model_training.ipynb	Naive Bayes and LSTM training and comparison
kafka_spark_pipeline/kafka_spark_streaming_colab.ipynb	Kafka and Spark streaming pipeline
kafka_spark_pipeline/dashboard/elasticsearch_dashboard_colab.ipynb	Elasticsearch storage and Plotly dashboard
kafka_spark_pipeline/*.pickle, *.keras	Trained model artifacts
reports/	Final report, comparison CSVs, charts, dashboard HTML files
presentation_slides.pptx	Presentation deck
requirements.txt	Python dependencies

Appendix B: Key Software Versions

Component	Version
Python	3.x (Google Colab)
Apache Kafka	3.9.1 (KRaft mode)
Apache Spark / PySpark	3.5.1
Elasticsearch	8.11.3
scikit-learn	1.9.0
TensorFlow / Keras	Keras 3
NLTK	for preprocessing
Plotly	for dashboard visualisation

Appendix C: Reproducibility Notes

- All random steps (train and test split, model start values) use a fixed random seed of 42 so the results can be repeated.
- The Kafka, Spark, and Elasticsearch notebooks are made to run from top to bottom in a fresh Google Colab session, with checkpoint notes that show the expected output at each stage.
- Model files are saved to disk so the streaming pipeline can load the exact trained model without training again.